L11 - 数据处理流水线

> *"数据是模型的食粮"*

📌 本节目标

1. 理解 LLM 不同训练阶段的数据格式（预训练、SFT、DPO）
2. 掌握 Dataset 和 DataLoader 的工作原理
3. 理解 padding、truncation 和 max_seq_len 的含义
4. **重点掌握 Loss Mask 机制**——预训练和 SFT 的核心区别之一
5. 理解 batch_size 与 gradient_accumulation_steps 的关系

📚 前置知识

- Python 基础，了解 JSON/JSONL 格式
- PyTorch 基础（Tensor 操作）
- 了解 Tokenizer 的工作原理（参考 L10）

---

1. LLM 训练数据的格式

训练 LLM 不同阶段需要不同格式的数据。可以把它想象成烹饪——不同菜式需要不同的食材加工方式。

1.1 预训练数据格式

预训练数据是最"原始"的——就是一段一段的纯文本。MiniMind 使用 JSONL 格式（每行一个 JSON 对象）：

```
{"text": "人工智能（Artificial Intelligence，简称AI）是计算机科学的一个分支..."}
{"text": "量子力学是物理学的一个基本理论，描述了微观粒子的行为..."}
```

每一行就是一个训练样本。模型的任务是：**给定前面的 token，预测下一个 token**（Next Token Prediction）。

MiniMind 的预训练数据集： | 数据集 | 文件名 | 大小 | 说明 | |--------|--------|------|------| | 小规模 | `pretrain_t2t_mini.jsonl` | 1.2GB | 快速实验用 | | 全量 | `pretrain_t2t.jsonl` | 10GB | 完整训练用 |

1.2 SFT（监督微调）数据格式

SFT 数据是**多轮对话**格式，让模型学会"你问我答"：

```
{
  "conversations": [
    {"role": "system", "content": "你是一个有用的助手。"},
    {"role": "user", "content": "什么是机器学习？"},
    {"role": "assistant", "content": "机器学习是人工智能的一个子领域..."}
  ]
}
```

关键区别：SFT 数据有明确的**角色划分**（system/user/assistant），模型需要学会根据用户输入生成合适的回复。

MiniMind 的 SFT 数据集： | 数据集 | 文件名 | 大小 | 说明 | |--------|--------|------|------| | 小规模 | `sft_t2t_mini.jsonl` | 1.6GB | 快速实验用 | | 全量 | `sft_t2t.jsonl` | 14GB | 完整训练用，包含 Tool Call 数据 |

1.3 DPO（偏好对齐）数据格式

DPO 数据包含"好回答"和"坏回答"的对比：

```
{
  "chosen": [
    {"role": "user", "content": "解释一下量子纠缠"},
    {"role": "assistant", "content": "量子纠缠是一种量子力学现象...（准确详细的回答）"}
  ],
  "rejected": [
    {"role": "user", "content": "解释一下量子纠缠"},
    {"role": "assistant", "content": "量子纠缠就是两个粒子心灵感应...（不准确的回答）"}
  ]
}
```

MiniMind 的对齐数据集： | 数据集 | 文件名 | 大小 | |--------|--------|------| | DPO | `dpo.jsonl` | 53MB | | RLAIF | `rlaif.jsonl` | 24MB |

---

2. Dataset 和 DataLoader

2.1 Dataset：数据的"仓库"

PyTorch 的 `Dataset` 类定义了数据集的结构。你需要实现两个核心方法：

```
class PretrainDataset(Dataset):
    def __init__(self, data_path, tokenizer, max_length):
        # 加载数据
        self.data = []
        with open(data_path, 'r') as f:
            for line in f:
                self.data.append(json.loads(line))
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        text = self.data[idx]['text']
        # 将文本转换为 token ids
        token_ids = self.tokenizer.encode(text)
        # 截断到 max_length
        token_ids = token_ids[:self.max_length]
        return token_ids
```

`__getitem__` 每次返回一个样本，这就像是从仓库里取出一箱食材。

2.2 DataLoader：数据的"传送带"

`DataLoader` 负责把 Dataset 中的数据**批量**送给模型：

```
dataloader = DataLoader(
    dataset,
    batch_size=32,          # 每批 32 个样本
    shuffle=True,           # 打乱顺序
    num_workers=4,          # 4 个进程并行加载
    collate_fn=collate_fn   # 自定义批量整合函数
)
```

为什么需要 `collate_fn`？因为不同样本的长度可能不同，需要统一处理（padding）。

---

3. Padding 和 Truncation 策略

3.1 为什么需要 Padding?

GPU 进行并行计算时，要求同一 batch 内所有序列长度相同。但自然语言文本长度参差不齐，所以需要 padding——用特殊的 pad token 填充短序列。

```
样本 1: [我, 喜, 欢, 学, 习, AI]          → 长度 6
样本 2: [今, 天, 天, 气, 好]              → 长度 5
样本 3: [机, 器, 学, 习, 是, 一, 个, 领, 域] → 长度 9

Padding 后（max_length=9）:
样本 1: [我, 喜, 欢, 学, 习, AI, PAD, PAD, PAD]
样本 2: [今, 天, 天, 气, 好, PAD, PAD, PAD, PAD]
样本 3: [机, 器, 学, 习, 是, 一, 个, 领, 域]
```

3.2 Truncation（截断）

当文本超过 `max_seq_len` 时，直接截断：

```
token_ids = token_ids[:max_seq_len]
```

3.3 max_seq_len 的含义与选择

`max_seq_len`（最大序列长度）是模型能处理的最长 token 序列。MiniMind 默认 `max_seq_len=512`。

选择原则： - **太短**：信息被截断，模型无法学到长距离依赖 - **太长**：显存占用剧增（Attention 的计算复杂度是 $O(n^2)$），训练变慢 - **经验法则**：覆盖 95% 以上的样本长度即可

---

4. Loss Mask：预训练与 SFT 的关键区别

这是面试高频考点，请务必理解透彻。

4.1 什么是 Loss Mask?

Loss Mask 是一个与 token 序列等长的 0/1 向量，用来**控制哪些 token 参与 loss 计算**。

- `1` 表示该位置的 loss 参与计算
- `0` 表示忽略该位置的 loss

4.2 预训练的 Loss Mask

预训练阶段，**所有 token 都参与 loss 计算**（除了 padding）：

```
文本:      [我, 喜, 欢, 学, 习, PAD, PAD]
Loss Mask: [1,  1,  1,  1,  1,  0,   0  ]
```

这很合理——预训练的目标是学习整个语言的统计规律，每个 token 都有价值。

4.3 SFT 的 Loss Mask（重点！）

SFT 阶段，**只计算 assistant 回复部分的 loss**，prompt（system + user）部分的 loss 被 mask 掉：

```
Token序列: [<|im_start|>, system, ..., <|im_end|>, <|im_start|>, user, 什么是AI?,
<|im_end|>, <|im_start|>, assistant, AI是..., <|im_end|>]
Loss Mask: [0,          0,      ..., 0,          0,          0,    0, ..., 0, 0,
0,         1, ..., 1,         1        ]
```

4.4 为什么 SFT 只计算回答部分的 loss?

这是面试必考题。核心原因：

1. **训练目标不同**：SFT 的目标是让模型学会"如何回答"，而不是学会"如何提问"
2. **避免"学歪"**：如果也计算 prompt 的 loss，模型可能会倾向于生成像用户提问的文本
3. **信号纯度**：只监督回答部分，梯度信号更纯净，训练更高效
4. **类比**：就像考试只判答案对不对，不判你看题看得对不对

4.5 Loss Mask 的代码实现

```python
def create_sft_loss_mask(input_ids, tokenizer):
    """为SFT创建loss mask，只在assistant回复部分计算loss"""
    loss_mask = torch.zeros_like(input_ids, dtype=torch.float)
    in_assistant = False

    for i, token_id in enumerate(input_ids):
        if is_assistant_start(token_id, tokenizer):
            in_assistant = True
            continue
        if is_turn_end(token_id, tokenizer):
            if in_assistant:
                loss_mask[i] = 1.0  # <|im_end|> 也计入 loss
            in_assistant = False
            continue
        if in_assistant:
            loss_mask[i] = 1.0
```

```
    return loss_mask
```

---

5. Batch Size 与梯度累积

5.1 为什么需要梯度累积?

理想的 batch size 可能很大（比如 256），但 GPU 显存有限，一次只能放下 batch_size=16。怎么办?

梯度累积（Gradient Accumulation）：把一个大 batch 拆成多个小 batch，累加梯度后再更新参数。

有效 batch size = batch_size × gradient_accumulation_steps × num_gpus

例如：batch_size=16，gradient_accumulation_steps=8，单卡训练：

有效 batch size = 16 × 8 × 1 = 128

5.2 梯度累积的实现

```
for step, batch in enumerate(dataloader):
    loss = model(batch)
    loss = loss / gradient_accumulation_steps  # 除以累积步数，保证梯度量级一致
    loss.backward()  # 累加梯度

    if (step + 1) % gradient_accumulation_steps == 0:
        optimizer.step()   # 更新参数
        optimizer.zero_grad()  # 清零梯度
```

关键点：`loss / gradient_accumulation_steps` 这一步**必不可少**。如果不除，累积的梯度会比正常大 N 倍，相当于放大了学习率。

5.3 Batch Size 选择的影响

Batch Size	优点	缺点
过小	显存友好，正则化效果	训练不稳定，收敛慢
过大	训练稳定，梯度估计准确	显存大，泛化可能差
适中	平衡速度和效果	需要实验确定

---

6. 数据清洗和去重

高质量的数据是训练好模型的前提。常见的数据处理步骤：

1. **去重**：删除完全相同或高度相似的样本（MinHash、SimHash）
2. **过滤**：去除低质量文本（过短、乱码、广告）
3. **敏感信息清理**：去除个人隐私信息
4. **语言过滤**：保留目标语言的文本
5. **格式标准化**：统一编码、换行符等

MiniMind 的训练数据已经经过清洗和去重处理，可以直接使用。在实际工程中，数据清洗通常占据整个项目 60% 以上的时间。

---

7. MiniMind 源码解读

7.1 关键文件

- `dataset/` 目录：存放所有训练数据
- `trainer/train_pretrain.py`：预训练数据加载逻辑
- `trainer/train_full_sft.py`：SFT 数据加载逻辑
- `trainer/train_dpo.py`：DPO 数据加载逻辑

7.2 数据加载流程

MiniMind 的数据加载流程大致如下：

```
JSONL文件 → 逐行读取JSON → Tokenizer编码 → Padding/Truncation → 构造Loss Mask → DataLoader批量输出
```

在 `train_pretrain.py` 中，预训练数据的处理：

```
# 伪代码，展示核心逻辑
def process_pretrain_sample(text, tokenizer, max_length):
    token_ids = tokenizer.encode(text)
    # 构造输入和标签：标签就是输入右移一位
    input_ids = token_ids[:-1]
    labels = token_ids[1:]
    # loss_mask 全部为 1（所有token都参与loss计算）
    loss_mask = [1] * len(labels)
    return input_ids, labels, loss_mask
```

在 `train_full_sft.py` 中，SFT 数据的处理：

```
# 伪代码，展示核心逻辑
def process_sft_sample(conversations, tokenizer, max_length):
    full_tokens = []
    loss_mask = []
    for turn in conversations:
        role = turn['role']
        content_tokens = tokenizer.encode(turn['content'])
        full_tokens.extend(content_tokens)
        if role == 'assistant':
            loss_mask.extend([1] * len(content_tokens))  # assistant 部分计入 loss
        else:
            loss_mask.extend([0] * len(content_tokens))  # 其他部分不计入 loss
    return full_tokens, loss_mask
```

🎤 面试考点

Q1: 为什么 SFT 不计算 prompt 部分的 loss?

答：SFT 的目标是训练模型生成高质量回复，而不是学会提问。如果计算 prompt 的 loss，模型的梯度信号会被"如何生成提问"的信息干扰，降低学习效率。只在 assistant 部分计算 loss，能让梯度信号更加纯净，模型更快学会"如何回答"。

Q2: Padding 放在左边还是右边？有什么区别?

答： - **训练时**通常用右 padding（在序列末尾填充），因为 causal attention mask 从左到右，右边填充不影响前面 token 的计算 - **推理时**做 batch inference 可能用左 padding，确保最后一个有效 token 对齐，便于 generation - MiniMind 训练时使用右 padding

Q3: 什么是梯度累积？为什么要除以 accumulation_steps?

答：梯度累积是将多个 mini-batch 的梯度累加后再更新参数的技术，用于在显存不足时模拟大 batch 训练。必须除以 accumulation_steps 是因为 `loss.backward()` 会把梯度**加到**现有梯度上，如果不做归一化，累积后的梯度会是正常值的 N 倍，相当于隐式地放大了 N 倍学习率。

Q4: max_seq_len 设置过大会有什么问题?

答： 1. 显存占用急剧增加：Self-Attention 的计算和存储复杂度为 $O(n^2)$ 2. 训练速度下降 3. 大部分样本可能远短于 max_seq_len，导致大量无效的 padding 计算 4. 可能引入位置编码外推的问题

Q5: 预训练和 SFT 的数据处理有何不同?

答： - **预训练**：数据是纯文本，所有 token 都参与 loss 计算，任务是无差别的 Next Token Prediction - **SFT**：数据是多轮对话格式，需要应用 chat_template 格式化，使用 Loss Mask 只在 assistant 回复部分计算 loss

Q6: DPO 数据和 SFT 数据有什么区别?

答：SFT 数据只有一个"标准答案"，模型通过模仿学习。DPO 数据包含 chosen（好回答）和 rejected（坏回答）的对比对，模型通过对比学习，学会区分好坏回答，从而进行偏好对齐。

✅ 自测题

1. 请画出从原始 JSONL 文件到 DataLoader 输出一个 batch 的完整流程图。
2. 给定以下对话，写出 Loss Mask： `System: 你是助手 User: 1+1=? Assistant: 等于2`
3. 如果 batch_size=8，gradient_accumulation_steps=4，使用 2 张 GPU（DDP），有效 batch size 是多少?
4. 为什么说"数据质量比数据数量更重要"？请结合 SFT 阶段举例说明。
5. padding token 应该参与 loss 计算吗？为什么?

⏭️ 下一节预告

L12 - 预训练 Pretrain：数据准备好了，接下来我们要用它来"喂养"模型了！我们将深入理解预训练的目标函数（交叉熵损失）、学习率调度策略、混合精度训练等关键技术，并跟读 MiniMind 的 `train_pretrain.py` 源码。